

VoiceGPT With Multilingual Facility

ASHWINI KUMAR SINHA

Earlier, we used the power of ChatGPT and converted it into VoiceGPT, making it the most advanced voice assistant that can talk. We ended the project with a promise to come up with a more advanced feature and release its next version. Today, we are excited to introduce the multilingual facility to our VoiceGPT.

That's right, now you can talk to ChatGPT in your own language, as it supports more than 100 languages. Unlike Alexa, you can now have a smart assistant that can communicate with you in a wide range of languages. Fig. 1 showcases the author's setup for testing the project. The components used in this project are listed in Table 1 for the Bill of Material. Simply gather all the components mentioned in Table 1 and interconnect them using the provided slots, eliminating the need for a circuit diagram.

ChatGPT voice control setup

To begin, you need to perform the initial setup of OpenAI and obtain the API key for OpenAI, as described in previous issue of this magazine. Once you have set up an AI account, you may choose to add billing based on your usage and requirements. With that complete, you can now delve into the code.

We have developed two versions of the multilingual code: a manual

version where you enter the desired language code for your voice assistant, and an automatic version that detects the language you input and uses that same language in VoiceGPT to communicate with you. Let's proceed with the coding.

Code for talking to ChatGPT

First, you need to import the speech recognition for NLP as well as GTTS. Alternatively, OpenAI Whisper can be used. Next, set the model

for ChatGPT that you are going to utilise. Since we are supporting multiple languages, we will be using the "text-davinci-003" model. You then set the OpenAI API.

We have created the chatGPT function, where we define the chatGPT query prompt, model to use, temperature, and other relevant settings. You can modify these values as per your requirements. Fig. 2 showcases a code snippet that sets the API and GPT model name.

Next, you create the loop function, which continuously checks the microphone and captures the human voice while filtering out background noise. Here comes the important part where you set

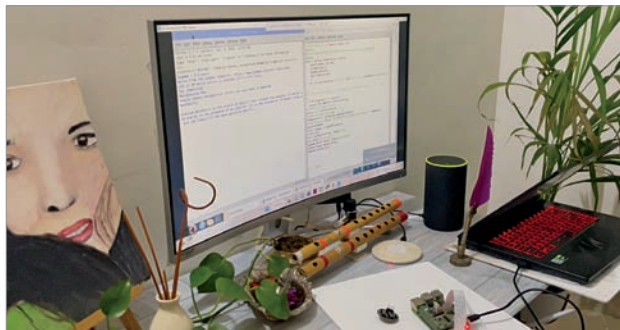


Fig. 1: Author's setup for testing the project

```
File Edit Format Run Options Window Help

import speech_recognition as sr
import math
import time
import sys
import openai
import pygame
from gtts import gTTS
pygame.mixer.init()
import pyttsx3
from langdetect import detect

#model_to_use="text-davinci-003" # most capable
#model_to_use="text-curie-001"
#model_to_use="text-babbage-001"
model_to_use="text-davinci-003" # lowest token cost
language_code= "hi"
r = sr.Recognizer()
#openai.api_key="sk-oIGZ7anMRPEVDYaYp2QZT3B1bkFJqenpx3ePg7osq7b3kMa3"
#openai.api_key="sk-cUxqAbooiQ30AoyTvmidT3B1bkFJm17vcfuifVMgiIhQoxjI"
openai.api_key="sk-m6b5oZyPD5RRJ3PR1BhxT3B1bkFJva5QEhPINhNipyQ7WJ1f"

def chatGPT(query):
    response = openai.Completion.create(
        model=model_to_use,
        prompt=query,
        temperature=0.7,
```

Fig. 2: Code snippet setting the API and GPT model name

BILL OF MATERIALS

Components	Quantity	Description
Raspberry Pi 4	1	1GB
USB MIC	1	Microphone
Speaker	1	For audio out
5V DC adaptor	1	5V 2A